CSE370 : Database Systems
Project Report
Project Title : BRACU Gig Management System

Group No : 05, CSE370 Lab Section : 11, Spring 2026		
ID	**Name**	**Contribution**
21301114	Md Sanzim Rahman Khan	Forum UI, Database Setup, Profile, Core Configuration, community forum
22201166	MD Ali Razin	Credit System, Authentication
22201655	MD INZAMAMUL HAQUE	Gig Management, Analytics Integration, Messaging Backend

Table of Contents

Section No	Content	Page No
1	Introduction	3
2	Project Features	4
3	ER/EER Diagram	5
4	Schema Diagram	6
5	Normalization	7-8
6	Frontend Development	9
7	Backend Development	10
8	Source Code Repository	11
9	Conclusion	11
10	References	11

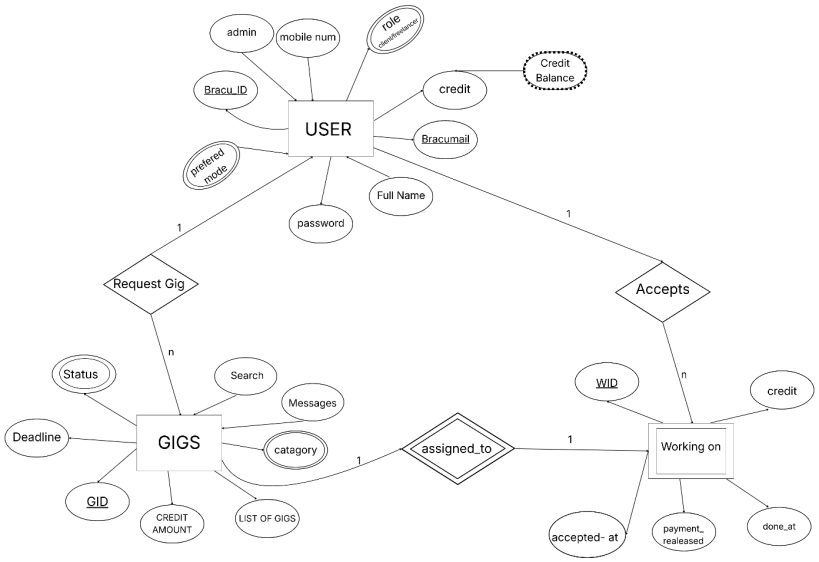
Introduction

This project involves the development of a web-based freelance and gig marketplace. The platform leverages a robust database system to manage user accounts, a credit-based transactional system for payments, persistent messaging for communication, analytics tracking for performance insights, and an integrated community forum to foster user engagement.

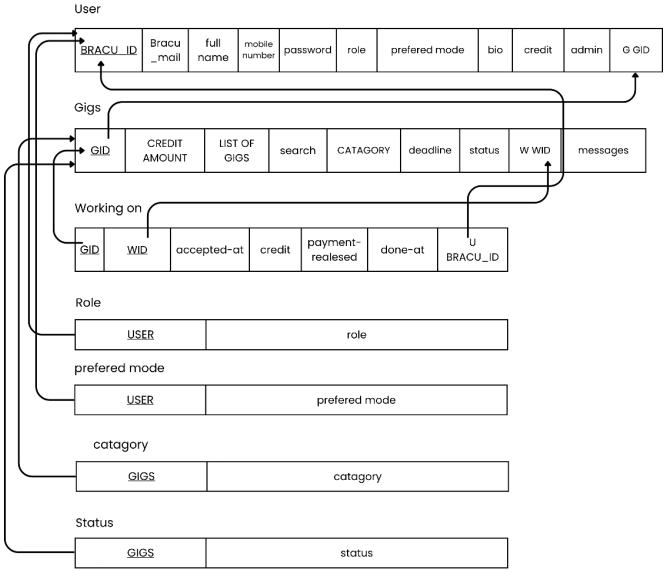
Project Features

ID, Name	Features [3 per member]	
21301114, Md Sanzim Rahman Khan	Ft 1:	Forum UI development, message views and profile pages.
	Ft 2:	Database setup for forum and sample data loading.
	Ft 3:	Core site configuration and helper modules. Community Forum.
22201166, MD Ali Razin	Ft 1:	UI integration for credit and profile management.
	Ft 2:	Credit system implementation, migrations, and verification scripts.
	Ft 3:	Search Gigs
22201655, MD INZAMAMUL HAQUE	Ft 1:	Analytics UI components and client/freelancer integration.
	Ft 2:	Messaging persistence backend and authentication improvements.
	Ft 3:	Gig Management

ER/EER Diagram



Schema Diagram



Normalization

a. Explain if your converted Schema is in 1NF or not. If not, decompose it to 1NF.

No, the converted schema is not in 1NF. A schema in 1NF must only contain atomic (indivisible) values, meaning no multivalued attributes or repeating groups. In your schema, the User table still contains multivalued attributes like role and preferred mode. Similarly, the Gigs table contains category, status, LIST OF GIGS, and messages.

To decompose it to 1NF, we must remove these from the main tables and rely entirely on separate mapping tables:

1. User (BRACU_ID [PK], Bracu_mail, full name, mobile number, password, bio, credit, admin, G_GID)
2. Gigs (GID [PK], CREDIT AMOUNT, search, deadline, W_WID)
3. Working on (WID [PK], GID [FK], accepted-at, credit, payment-released, done-at, U_BRACU_ID [FK])
4. User_Role (BRACU_ID [FK], role)
5. User_Preferred_Mode (BRACU_ID [FK], preferred mode)
6. Gig_Category (GID [FK], category)
7. Gig_Status (GID [FK], status)
8. Gig_Messages (GID [FK], message_id, message_text

b. Explain if your converted Schema is in 2NF or not. If not, decompose it to 2NF. Can there be any partial functional dependencies in your relational schema?

No, the schema is not in 2NF. Yes, there are partial functional dependencies caused by how the 1-to-N "Request Gig" relationship was mapped.

By placing the foreign key G_GID inside the User table, a user requesting multiple gigs would require a composite primary key (BRACU_ID, G_GID). This creates a partial dependency because user details like full name and mobile number would only depend on BRACU_ID (part of the key), not the whole key. To fix this and decompose to 2NF, we must move the foreign key to the "N" side (the Gigs table):

1. User (BRACU_ID [PK], Bracu_mail, full name, mobile number, password, bio, credit, admin)
2. Gigs (GID [PK], CREDIT AMOUNT, search, deadline, W_WID, BRACU_ID [FK]) (Added BRACU_ID to represent the gig requester)
3. Working on (WID [PK], GID [FK], accepted-at, credit, payment-released, done-at, U_BRACU_ID [FK])
4. (All 1NF mapping tables from part 'a' remain the same)

c. Explain if your converted Schema is in 3NF or not. If not, decompose it to 3NF. Can there be any transitive dependencies in your relational schema?

No, the schema is not in 3NF. Yes, there are transitive dependencies present in the design. A transitive dependency occurs when a non-prime attribute depends on another non-prime attribute.

For example, Credit Balance is a derived attribute that depends on the credit attribute and transactions; it should be calculated, not stored. Additionally, if the admin status is determined by what is in the role table, that is a transitive dependency. Finally, having W_WID in Gigs and GID in Working on creates a redundant loop that can cause update anomalies. To decompose to 3NF, we remove these dependencies:

- User (BRACU_ID [PK], Bracu_mail, full name, mobile number, password, bio, credit) (Removed 'admin' assuming it's handled by the User_Role table, and ensure 'Credit Balance' is never stored as a column)
- Gigs (GID [PK], CREDIT AMOUNT, search, deadline, BRACU_ID [FK]) (Removed W_WID to break the redundant loop)
- Working on (WID [PK], GID [FK], accepted-at, credit, payment-realesed, done-at, U_BRACU_ID [FK])
- (All 1NF mapping tables from part 'a' remain the same

Frontend Development

The frontend development efforts focused on creating user interface pages for three key user groups: clients, freelancers, and the community. This included the implementation of core UI components for profile management, messaging, and forum interaction. A significant part of the work involved styling and layout adjustments, as well as integrating components for site-wide functionalities like analytics and search.

Contribution of ID : 21301114, Name : Md Sanzim Rahman Khan

Developed core community-facing UIs, including the forum structure, comprehensive message views, and user profile/rating pages. Additionally, this contribution covered various client and freelancer pages and critical site-wide includes (e.g., header, footer, search) that govern the overall visual structure and search functionality. (Fig : 1.1 and Fig : 1.2)

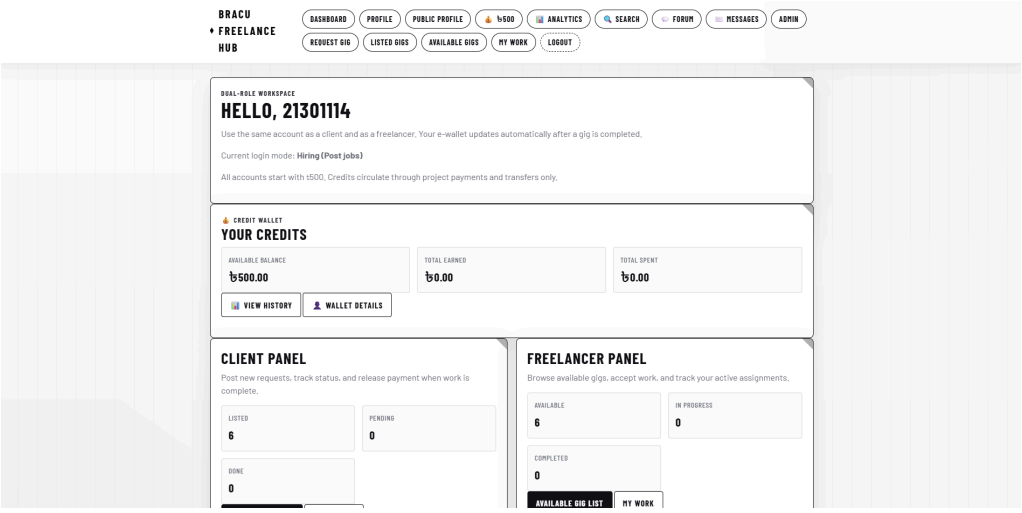


Fig : 1.1

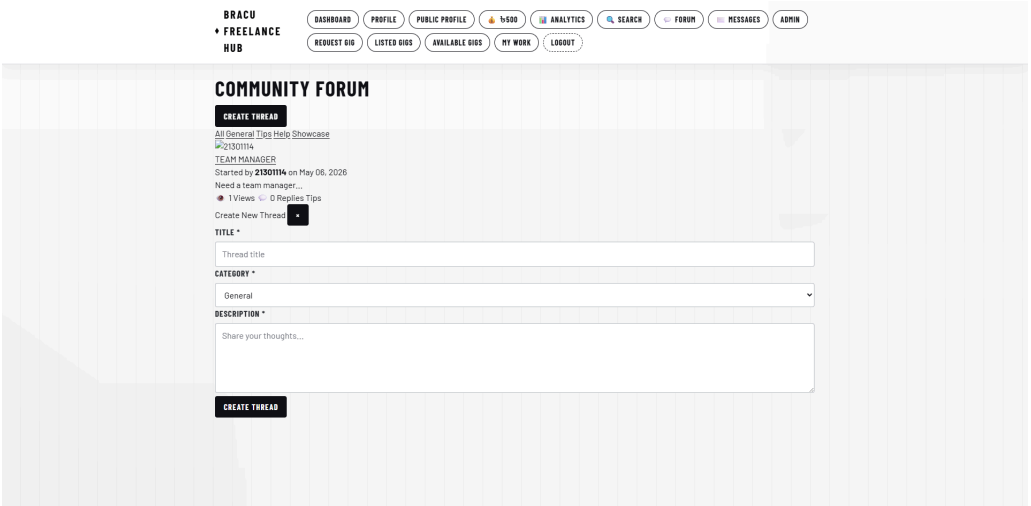


Fig : 1.2

Contribution of ID : 22201166, Name : MD Ali Razin

Focused on integrating key UI features related to the credit system, user profile management, and direct messaging across pages like gig creation and listing views. This work also involved maintaining the visual consistency of the application through minor styling and layout updates in the main stylesheet.

Contribution of ID : 22201655, Name : MD INZAMAMUL HAQUE

Specialized in implementing the analytics user interface, integrating visual components onto both client and freelancer dashboards. This required coordination with internal specifications for feature completion.

Backend Development

The backend development established the core architectural functionality of the system, covering essential areas such as user authentication, database management, transactional systems, communication persistence, and data analytics hooks. This involved setting up configuration, migration scripts, and helper modules for application logic.

Contribution of ID : 21301114, Name : Md Sanzim Rahman Khan

Established the foundational database structure and data management scripts for the community forum, including primary setup and scripts for populating sample data. Core site functionality was ensured through the setup of general site configuration, database connection handlers, and shared utility functions. (Fig : 2.1, Fig : 2.2, Fig : 2.3 and Fig : 2.4)

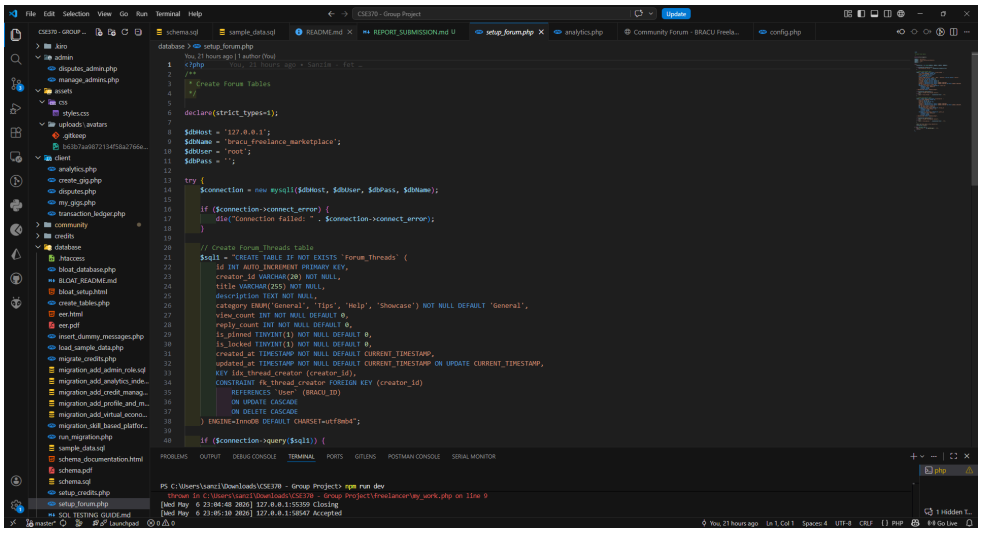


Fig : 2.1

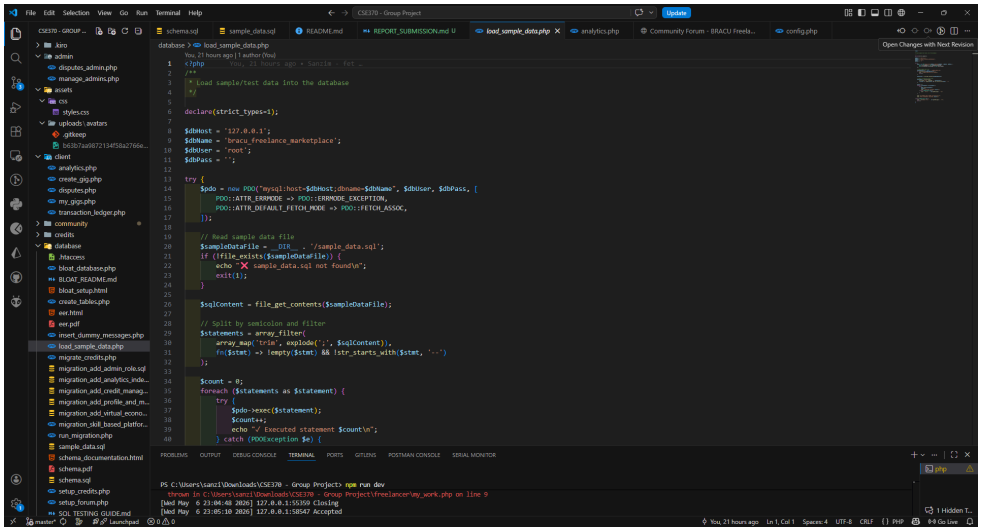


Fig : 2.2

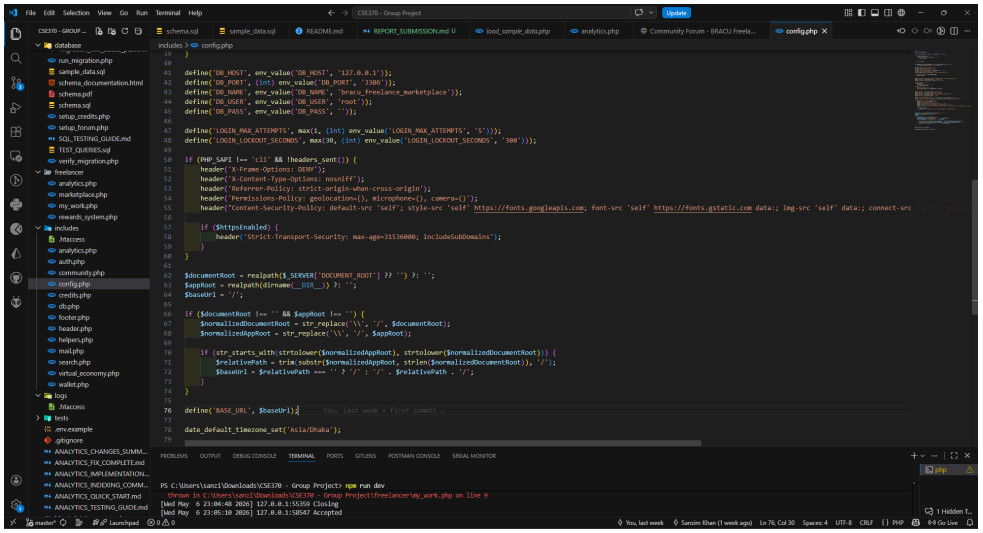


Fig : 2.3

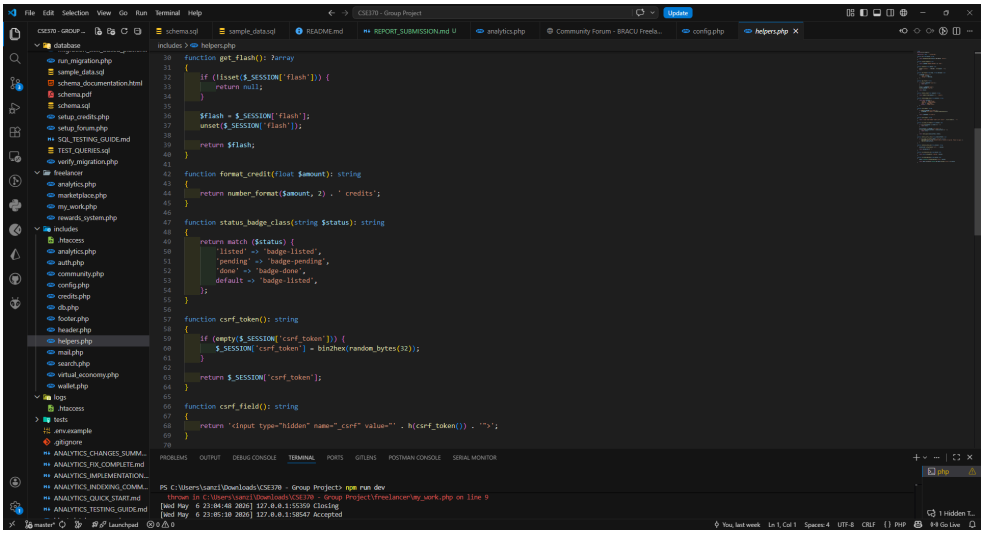


Fig : 2.4

Contribution of ID : 22201166, Name : MD Ali Razin

Implemented the credit management system, including necessary database migration scripts and the core credit logic module. This contribution also included developing the backend for messaging persistence and creating scripts to seed dummy messages. Essential security and access control were managed through improvements to the authentication module and the user registration process.

Contribution of ID : 22201655, Name : MD INZAMAMUL HAQUE

Responsible for integrating the core analytics back-end functionality. This work was thoroughly documented in dedicated markdown files (e.g., ANALYTICS_IMPLEMENTATION.md and ANALYTICS_QUICK_START.md) and validated through dedicated unit tests for both analytics and credit system components.

Source Code Repository

The project source code is publicly available on GitHub at https://github.com/SanzimKhan/CSE370---Group-Project (master branch).

Conclusion

This report confirms the division of labor and lists the specific individual contributions derived from the project's commit history. The frontend team successfully delivered key client, freelancer, and community pages alongside styling and search/analytics integration. The backend team established the system's foundation, completing critical work on authentication, the credit system, messaging persistence, database migrations, and analytics data hooks.

References

Project Repository: https://github.com/SanzimKhan/CSE370---Group-Project
Commit log (local repository).